



동적 메모리 할당의 응용

01_ 동적 메모리 할당의 기본

02_ 동적 메모리 할당의 응용

Static vs Dynamic memory allocation

```
int A[100];  
A[99] = 5000;
```

```
int * B;  
int n;  
cin >> n;  
B = new int[n];  
B[89] = 50; // *(B+89) = 50;
```

- 메모리의 크기를 프로그램이 실행되기 전(compile time)에 결정
- 메모리의 크기를 실행하는 도중(run time)에 결정

기본적인 동적 메모리 할당과 해제(1)

- 사용자에게 입력받은 정수의 합과 평균을 구하는 예

```
// 몇 개의 정수를 입력할지 물어본다.
int size;
cout << "몇 개의 정수를 입력하시겠습니까? ";
cin >> size;

// 필요한 만큼의 메모리를 할당한다.
int* arr = new int [size];

// 정수를 입력받는다.
cout << "정수를 입력하십시오.\n";
for (int i = 0; i < size; ++i)
    cin >> arr[i];

// 평균을 계산하고 출력한다.
int sum = 0;
for (i = 0; i < size; ++i)
{
    sum += arr[i];
}
float ave = (float)sum / (float)size;
cout << "합 = " << sum << ", 평균 = " << ave << "\n";

// 사용한 메모리를 해제한다.
delete[] arr;
```



기본적인 동적 메모리 할당과 해제(2)

- 실행 결과

[16-3]

- 동적으로 메모리를 할당하기

[16-4]



기본적인 동적 메모리 할당과 해제(3)

- 메모리를 할당했을 때의 메모리 상태

Stack memory

Heap memory

[16-7]

- 동적으로 할당한 메모리를 해제하기

[16-8]



기본적인 동적 메모리 할당과 해제(4)

- 동적 메모리 할당을 사용하는 방법의 정리
 - 메모리를 할당할 때는 타입과 크기를 지정한다. 그러면 컴퓨터가 메모리를 할당한 후에 그 메모리의 주소를 보관한다. 우리는 이 주소를 보관해두어야 한다.
 - 보관해둔 주소를 통해서 메모리⁶공간을 사용할 수 있다. 이 때는 배열의 원소를 가리키는 포인터처럼 사용할 수 있다.
 - 사용이 끝난 후에는 반드시 보관해 둔 주소를 알려주면서 메모리를 해제한다.



연산자 new, delete, new[], delete[]

- 변수 하나를 동적으로 할당하기 위해서 new, delete를 사용한다.

```
// int 하나를 담을 수 있는 크기의 메모리 공간을 할당한다.  
int* p = new int;  
  
// 메모리에 값을 넣어본다.  
*p = 337;  
  
// 사용이 끝난 메모리를 해제한다.  
delete p;
```

동적 메모리 할당의 규칙(1)

- new, delete와 new[], delete[] 쌍을 맞춰서 사용하자.
- NULL 포인터를 해제하는 것은 안전하다.

```
char* p = NULL;
delete p;          // 혹은 delete[] p

// 다음과 같이 할 필요가 없다.
if ( NULL != p )
    delete p;
```

- 해제한 메모리를 또 해제해서는 안 된다.

```
// 메모리를 할당한다.
short* p = new short [100];

// 메모리를 해제한다.
delete[] p;

// 메모리를 해제한다.
delete[] p;      // Error
```

- 메모리를 해제한(delete)후에는 메모리를 가리키던 포인터의 값을 (NULL)로 초기화하는 습관!!!

동적 메모리 할당의 응용(1)

- 문자열을 뒤집어서 복사해주는 함수

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1]; // 널문자 삽입을 위해 -1 함

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original, 8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```



동적 메모리 할당의 응용(2)

- 실행 결과

[16-14]

- 프로그램의 구조

[16-15]

동적 메모리 할당의 응용(3)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original,
                               8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-16]

동적 메모리 할당의 응용(4)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original, 8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-17]

동적 메모리 할당의 응용(5)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original,
                                8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-18]

동적 메모리 할당의 응용(6)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[10] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original, 8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-19]

동적 메모리 할당의 응용(7)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original,
        8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-20]

동적 메모리 할당의 응용(8)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original,
                                8);
    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-21]

동적 메모리 할당의 응용(9)

- 화살표까지 실행되었을 때의 메모리 상태

```
char* ReverseString(const char* src, int len)
{
    char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original,
                                8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```

[16-22]



동적 메모리 할당대신 배열은?

- 문자열을 뒤집어서 복사해주는 함수

```
char* ReverseString(const char* src, int len)
{
    char reverse[100]; //char* reverse = new char [len + 1];

    for (int i = 0; i < len; ++i)
        reverse[i] = src[len - i - 1];

    reverse[len] = NULL;
    return reverse;
}

int main()
{
    char original[] = "NEMODORI";
    char* copy = ReverseString( original, 8);

    cout << original << "\n";
    cout << copy << "\n";

    delete[] copy;
    copy = NULL;
    return 0;
}
```